



CaSMap: Agile Mapper for Reconfigurable Spatial Architectures by Automatically Clustering Intermediate Representations and Scattering Mapping Process

Xingchen Man, Jianfeng Zhu, Guihuan Song, Shouyi Yin, Shaojun Wei, Leibo Liu*

Beijing National Research Center for Information Science and Technology (BNRist)

School of Integrated Circuits, Tsinghua University, Beijing, China

E-mail: liulb@tsinghua.edu.cn

ABSTRACT

Today, reconfigurable spatial architectures (RSAs) have sprung up as accelerators for compute- and data-intensive domains because they deliver energy and area efficiency close to ASICs and still retain sufficient programmability to keep the development cost low. The mapper, which is responsible for mapping algorithms onto RSAs, favors a systematic backtracking methodology because of high portability for evolving RSA designs. However, exponentially scaling compilation time has become the major obstacle. The key observation of this paper is that the key limiting factor to the systematic backtracking mappers is the waterfall mapping model which resolves all mapping variables and constraints at the same time using single-level intermediate representations (IRs).

This work proposes CaSMap, an agile mapper framework independent of software and hardware of RSAs. By clustering the lowest-level software and hardware IRs into multi-level IRs, the original mapping process can be scattered as multi-stage decomposed ones and therefore the mapping problem with exponential complexity is mitigated. This paper introduces (a) strategies for clustering low-level hardware and software IRs with static connectivity and critical path analysis. (b) a multi-level scattered mapping model in which the higher-level model carries out the heuristics from IR clustering, endeavors to promote mapping success rate, and reduces the scale of the lower-level model. Our evaluation shows that CaSMap is able to reduce the problem scale (nonzeros) by 80.5% (23.1%-94.9%) and achieve a mapping time speedup of 83× over the state-of-the-art waterfall mapper across four different RSA topologies: MorphoSys, HReA, HyCUBE, and REVEL.

CCS CONCEPTS

• **Computer systems organization** → **Reconfigurable computing**; • **Software and its engineering** → **Retargetable compilers**.

KEYWORDS

Reconfigurable Spatial Architecture, Coarse-Grained Reconfigurable Architecture, Compiler, Integer Linear Programming

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

ISCA '22, June 18–22, 2022, New York, NY, USA

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-8610-4/22/06.

<https://doi.org/10.1145/3470496.3527426>

ACM Reference Format:

Xingchen Man, Jianfeng Zhu, Guihuan Song, Shouyi Yin, Shaojun Wei, and Leibo Liu. 2022. CaSMap: Agile Mapper for Reconfigurable Spatial Architectures by Automatically Clustering Intermediate Representations and Scattering Mapping Process. In *The 49th Annual International Symposium on Computer Architecture (ISCA '22)*, June 18–22, 2022, New York, NY, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3470496.3527426>

1 INTRODUCTION

Nowadays, due to the compound effect of the decline of Moore's Law and the soaring demands for energy-efficient compute, domain-specific accelerators (DSAs) have proliferated in emerging data- and compute-intensive fields, such as machine learning, digital signal processing and bioinformatics. In the design of DSAs, reconfigurable spatial architectures (RSAs), including coarse-grained reconfigurable architectures (CGRAs), have been increasingly adopted for their potential performance, energy/area-efficiency, and flexibility after fabrication [34]. Nevertheless, to fulfill these potentials is not trivial, especially considering that the scales of algorithm and architecture are both boosting. Currently, a scalable compiler that efficiently maps ever-changing algorithms onto various RSAs is the paramount concern. The compiler for RSAs faces a major problem with its back-end mapper which is different from that of CPUs and GPUs in terms of spatially distributed, software-managed on-chip computation, communication, and storage and is also different from that of field programmable gate arrays (FPGAs) in terms of runtime reconfigurability. The objective of the mapper is to find a valid and efficient mapping between software IR and hardware IR which are generated by a front-end parser to describe the functions and structures of target algorithms and accelerators respectively (see Fig. 1). Since the mapping process can be only modeled as an NP-Complete problem such as maximum clique or subgraph isomorphism problem [16], a naive mapper design could lead to an exponentially scaling compilation time yet worse quality than that of a sophisticated one. The design of the mapper plays a determining role in the compiler for RSAs, which is the focus of this paper. The state-of-the-art designs of the mapper for RSAs fall in two main categories:

- **Heuristic searching** (e.g., [3, 23, 61, 64]) is an empirical approach to expedite the searching process of mappers at an acceptable loss of mapping quality. Developers must devote a great deal of effort to designing a fixed heuristic algorithm for a specific software-hardware combination. Although the

*Corresponding author: Leibo Liu (liulb@tsinghua.edu.cn)

metaheuristic algorithm is general-purpose, considerable efforts are still required to accommodate the algorithm parameters and functions, etc. Machine-learning-directed mapping is also portable but requires tuning. Therefore, the heuristic approach is efficient but suffers from high development cost.

- **Systematic backtracking** (e.g., [8, 11, 37, 41]) is a complete approach that formulates the mapping process as a constraint satisfactory problem with optional objective, which is advantageous in succinct expression, integrity, portability, and solution quality. The commercial off-the-shelf or open-source constraint solvers, such as Gurobi [42] and Boolector [39], can be adopted, drastically lowering the development cost.

In the area of RSAs, both the hardware structure and software stack are volatile and fast-evolving. Therefore, the systematic backtracking mapper which can be easily adjusted to a new architecture and language is becoming more attractive than the heuristic mapper [8, 11, 37, 41]. However, there remains an enormous obstacle with the feasibility of systematic backtracking mapper, which is the unaffordable compilation time as long as tens of hours for typical hardware and kernels on realistic scale [27, 52, 55, 63]. A portable and scalable mapper has been acknowledged as a major challenge in RSA [8, 17, 23, 57, 63].

We are inspired by an observation of the systematic backtracking mappers: there are so many trivial details within their models of hardware and software that the mappers waste most of the compilation time on searching insignificant yet formally necessary space (comprising similar and low-impact solutions). The prior systematic backtracking mappers [8, 11, 37, 41] all adopt the waterfall mapping model whose variables are strongly coupled with each other, like stages in the waterfall project management [45]. For example, in the integer linear programming (ILP) based mapper of CGRA-ME [8], each switch and router are modeled as plenty of variables in a modulo routing resource graph (MRRG) [36] model to generate a complete configuration while these circuitry substrates can be all abstracted off in a heuristic time-extended CGRA (TEC) model [16, 62], saving a lot of search efforts. To summarize, **our insight is that the waterfall mapping model (i.e., all variables and constraints that define mapping software operations and dependencies to hardware elements are resolved simultaneously using single-level intermediate representations) is the key limiting factor in prior systematic backtracking mappers.**

This paper addresses the above challenges by proposing an agile mapper named CaSMap which develops a novel mapping model to replace the waterfall mapping model in systematic backtracking mappers. This idea is inspired from the concept of agile software development [18] which reforms the conventional waterfall development method. We also draw lessons from a classic approach, multi-level optimization [53], to solving NP-Complete problems. The proposed mapping model of CaSMap mitigates the problem of exponentially scaling of variables and constraints by means of clustering the lowest-level software and hardware IRs into multi-level IRs. By automatic analysis on connectivity, importance, etc., trivial variables and constraints are abstracted off to construct the

higher-level variables and constraints of the clustered IRs. Therefore, the local mapping for the higher-level element is almost effortless. In addition, the original one-pass mapping process for lowest-level IRs can be scattered as multi-stage decomposed mapping processes, including mapping higher-level clusters first and mapping internal details afterwards with necessary iterations. As a result, the new mapping process becomes modular and incremental, and thus CaSMap can achieve superior scalability. Essentially, akin to heuristic methods, CaSMap introduces some heuristics, such as strongly connected hardware region and critical data-flow subgraph, to prune the search space of systematic backtracking. But the heuristics come from automatic static analysis of the input hardware and software IRs during mapping, not from priori expert knowledge, or dynamic random search, or excessive data profile. Therefore, CaSMap is as portable as the original systematic backtracking mappers. CaSMap provides a framework for designing agile mappers for RSAs, with a tool box of strategies for clustering hardware/software IRs and scattering mapping algorithms.

This paper makes the following key contributions:

- Our work is the first to create an agile mapper, CaSMap, for RSAs to the best of our knowledge. An agile mapper is a mapper that has multi-level self-contained incremental mapping models and multi-level software/hardware IRs. It can be constructed without cumbersome hyper-parameter tuning and architecture-specific searching algorithm design.
- We create a general framework that provides rich strategies of clustering IRs and scattering mapping algorithms for designing an agile mapper. With the combination of these strategies, CaSMap has higher success rate (consequently less iterations) and performance.
- We implemented and evaluated CaSMap for classic and latest RSA topologies: MorphoSys [47], HReA [33], HyCUBE [54], and REVEL [56]. The quality of mapping and the scalability of software/hardware are compared with the state-of-the-art systematic backtracking method. Evaluation results demonstrate 80.5% problem scale and 98.8% time cost reduction over the state-of-the-art waterfall mapper, AA-ILP[8], maintaining the same initiation intervals in the meanwhile.

2 BACKGROUND AND MOTIVATION

2.1 Reconfigurable Spatial Architectures

In stark contrast to previous out-of-order and SIMD/SIMT processors (e.g., DSP and GPU), RSAs, as shown in Fig. 1, eliminate complex instruction pipeline and centralized communication to achieve high energy efficiency [7, 32]. In recent years, Miscellaneous RSAs have begun to emerge as domain-specific accelerators such as Eyeriss [7] and FlexFlow [35] for artificial intelligence (AI) applications, Darwin-WGA [51] for genome alignments workloads, and HyCUBE [54] for internet of things (IOT). These architectures differ a lot in PE organization, interconnection topology, etc., because architects typically perform domain-specific design space exploration in the dimensions shown in Table 1. For the purpose of low research and development cost, it is necessary for architects to build efficient compilers for their own RSAs.

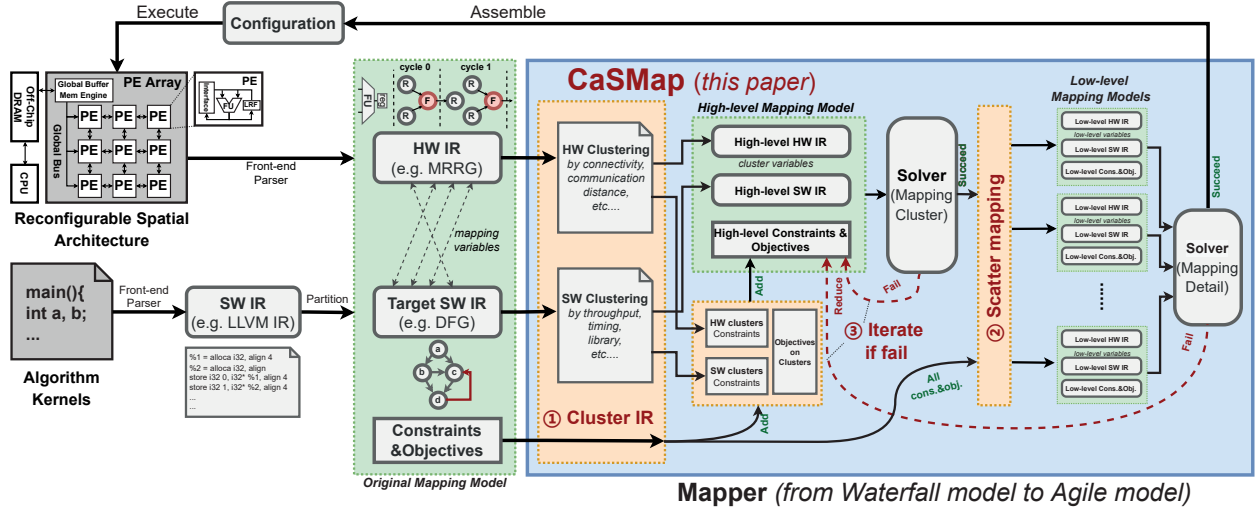


Figure 1: CaSMap framework plays an essential role in a portable compilation framework for reconfigurable spatial architectures. CaSMap contains two main phases: 1) Clustering software and hardware IRs and 2) Scattered mapping process.

Table 1: Design space of reconfigurable spatial architecture.

Dimensions	Design Space
Reconfiguration	Static, Dynamic
PE Function	Homogeneous, Heterogeneous, Dedicated, General-purpose
Interconnect Topology	Mesh-like, Crossbar, Point-to-point, Bus
On-Chip Buffer	Local, Global, On-chip Memory
Memory System	Scratchpad, Cache
Coupling to Main Processor	Tight, Loose

2.2 Reconfigurable Spatial Architecture Compilation Overview

To ensure portability and productivity, a good compilation framework should be independent of hardware and software, combining the architectures described with hardware construction language (HCL) and software kernels written in high-level programming language as inputs to generate machine code (configuration) ultimately. As illustrated in Fig. 1, after firstly parsing the hardware and software descriptions, the compiler constructs corresponding intermediate representations (IRs). Next, the mapper strives to find high-quality mapping between them promptly. Finally, the mapping result will be translated into binary machine code to be executed on RSAs. The following three principal parts of the compiler greatly affect the efficiency, portability, and development cost.

Software IR: Software IR is an equivalent form of the code written in high-level programming language and is language-agnostic, because it can be obtained from different compiler front-ends [6, 15] (e.g., LLVM-IR[26] can be obtained from language-specific LLVM-based compilers such as Clang [25] for C/C++/Objective-C, Unladen Swallow [60] for Python). Based on static single assignment (SSA) form which is adopted by widely used compilers, GCC and LLVM

[26], researchers also built data flow graph (DFG) or control data flow graph (CDFG) as IRs to analyze the original code.

Hardware IR: Hardware IR describes the structure of the target RSA, and is an abstraction layer that can be ported to different architectures [8, 46]. By abstracting basic functional and routing components and their input/output data ports as vertices, abstracting physical connections among them as edges, and replicating them along the time axis, a graph-based hardware IR can be set up (e.g., MRRG [36]).

Mapper: To keep every operation and dependency in software IR unchanged after mapping, the mapping problem is naturally modeled as an NP-Complete graph homomorphism problem[16]. To solve the problem, the state-of-the-art mappers adopt different searching approaches (shown in Table 2). Metaheuristics like genetic and simulated annealing algorithms are generic approaches for seeking a good solution in a large discrete searching space. However, they cannot derive rational heuristics from the mapping problem and require considerable manual efforts to tuning hyperparameters for specific architectures. The heuristic of machine-learning-directed mapping such as [29, 31, 61] can be automatically generated only after tedious modeling and training. RAMP and TAEM are mappers using heuristic algorithms designed for specific RSAs. Therefore, code needs to be rewritten when the architecture changes. Systematic backtracking methods such as integer linear programming (ILP) and satisfiability modulo theory (SMT) have become a popular solution because they are architecture-agnostic. However, they lack heuristics and result in unacceptable time costs. In fact, modern solvers already support adding custom heuristics for faster speed [12]. CaSMap is able to automatically add heuristics to the mapping model by clustering software and hardware IRs into multi-level IRs for agile mapping.

To sum up, heuristic-based mappers are fast but need more development cost. Systematic mappers have become a popular solution due to great portability but their speed is slow. An agile mapper is necessitated during researching and designing novel RSAs because

Table 2: State-of-the-art RSA Mappers vs CaSMap.

Mapper	Searching Method	Heuristic Type	Hyperparameters	HW IR	SW IR
GenMap[22] PolyMap[30]	Metaheuristic (Genetic)	Auto	Crossover Mutation Fitness, etc.	Architectural Parameters	DFG
DRESC[36] CGRA-ME (SA)[8]	Metaheuristic (Simulated Annealing)	Auto	Initial Temp Cooling Factor Final Temp, etc.	MRRG	DFG
DFGNet[61] RLMap[31] LISA[29]	Hyper-heuristic (Machine Learning)	Auto	NN Structure Training Set Learning Rate, etc.	TEC/MRRG	DFG
CGRA-ME (ILP)[9] Constraint-Centric [41]	Systematic (ILP)	None	None	MRRG	DFG
RAMP[10] TAEM[23]	Heuristic	Expertise	None	TEC	DFG
CaSMap(this work)	Systematic (ILP)	Auto	None	Multi-level	Multi-level

the performance and energy efficiency can be evaluated only after mapping [49, 55].

2.3 Insights

Most state-of-the-art RSA mappers formulate the above problem as searching an efficient mapping from every software IR element (operation and dependency) to every hardware IR element (functional unit, interconnection, and storage) within a group of constraints of routability, exclusiveness, timing and so on [8, 11, 37, 41]. In essence, the problem is resolved in a waterfall model which is linear, sequential, inflexible, and time-consuming. Inspired by the well-known agile software development methodology, a modular, incremental, and iterative model could be a more practical solution with both speed and quality, especially for fast-changing problems. What's more, a multi-level refinement method for combinatorial optimization problems [53] also gives us ideas, which involves recursive coarsening to create a hierarchy of approximations to the original problem and the initial solution is refined iteratively at each level. Our core idea is to transform the waterfall mapping model into an agile multi-level model supported by clustering software and hardware IRs into multi-level IRs. After solving the high-level small scale mapping model which is set up with high-level IRs, the mapping result can be conveyed to the lower mapping model by constraints generated by clustering. Therefore, the problem of exponentially scaling of variables and constraints can be mitigated. Interestingly, we find that the RSA is in nature suitable to our idea:

1) The RSA has a special structure composed of spatially distributed computation and storage units among which the communications are mostly localized because the RSA's interconnection is becoming more depopulated for energy efficiency purpose. As a result, the RSA is highly prone to form flexible local hardware regions that is self-contained and free to map any shapes of software IR fragments on.

2) The RSA has a unique computation model, which is a combination of temporal computing and spatial computing. The performance is usually determined by the pipelines and parallelism, meaning that only the software IR region containing throughput-critical paths should be mapped as a whole in order to search an optimal local solution that minimizes the path length. The other software IR regions could be arbitrarily divided and conquered in parallel in order to accelerate the searching. Although some repeaters or buffers are needed to construct a final valid mapping, there is little impact on performance most of the time thanks to the abundant parallel resources in RSAs.

3 CASMAP FRAMEWORK

To expedite the process of searching for an efficient mapping from software to hardware in a gigantic solution space and retain portability in the meantime, we develop CaSMap (agile mapper by automatically clustering IRs and scattering mapping process) for RSAs. CaSMap can mitigate application deployment and mapper development costs, furnishing rapid evaluation of software and hardware system of RSAs.

3.1 CaSMap Overview

As depicted in Fig. 1, CaSMap, which contains IR clustering strategies and scattered mapping process with multi-level mapping models. Before going deeper into the particulars, we firstly give a general introduction to CaSMap.

Original Software and Hardware IRs: As depicted in Fig. 1, CaSMap takes low-level software and hardware IRs as inputs. The word *low-level* here indicates that input IRs are fine-grained. More concretely, the low-level software IR is only comprised of basic operations and no sophisticated data structures except scalars. The low-level hardware IR is only made up of basic functional units (FUs), delay units, multiplexers, and wires among them. Taking low-level IRs as inputs offers advantageous portability to different languages and architectures.

Multi-level Software/Hardware IRs (Secs. 3.2&3.3): By clustering the basic elements in low-level IRs statically with customized strategies, high-level IRs can be created automatically and statically. High-level IRs are abstract layers that can imply the heuristics acquired from static clustering and facilitate constructing a cluster-level mapping model. The high-level and low-level IRs together form multi-level IRs. Users can adopt the clustering strategies introduced in this paper or develop their own strategies.

Multi-level Mapping Model (Sec. 3.4): Contrary to traditional systematic backtracking and heuristic searching approaches to mapping problems. CaSMap can automatically establish a multi-level mapping model by conveying the high-level mapping results to the low-level mapping models for the purpose of agile mapping. The variables, constraints, and objectives of the high-level mapping model are derived from the original mapping model and the coupling constraints brought by the organization of multi-level IRs.

Automatic Mapping Iteration (Sec. 3.5): The two high-level IRs may be incompatible to make the high-level mapping process fail. Moreover, the high-level mapping result may prune the solution space too much to make the low-level mapping fail as well. Therefore, CaSMap can iteratively tailor the constraints and objectives using multi-level IRs to acquire an appropriate multi-level mapping model automatically.

Before going deeper into the details of the framework, all the symbols that will appear in the rest of this section are listed in Table 3.

3.2 Hardware Clustering

There are mainly two kinds of programmable primitives when building RSAs: functional units (FUs) and connection units (CUs), which resemble operations and data dependencies respectively in the software description. The FU is the most basic component to support a set of arithmetic or logical operations and able to

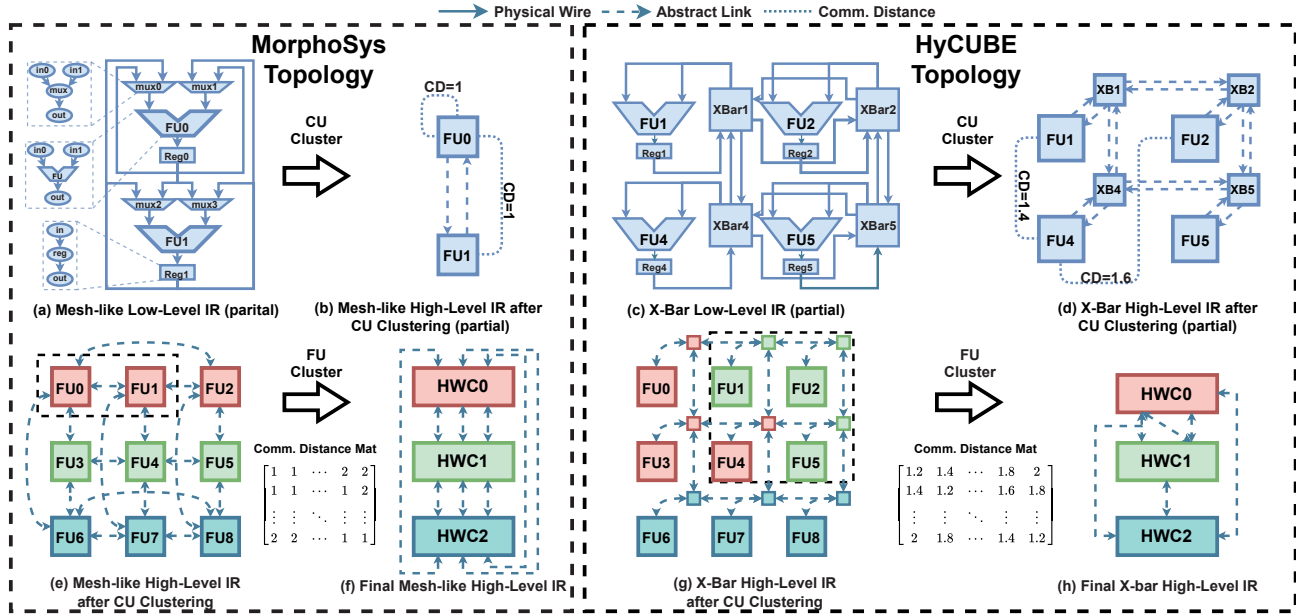


Figure 2: High-level hardware IRs can be obtained via a two-step clustering. Firstly, the physical wires in (a) and (c) are compressed to abstract links to form (e) and (g). Next, final high-level hardware IRs (f) and (h) are acquired by FU clustering.

Table 3: List of symbols for ILP model construction.

Symbol	Description
$D(V, E)$	The original DFG.
$G(V, E)$	The original MRG.
$f_{i,j}$	Binary variable: whether operator $i \in V(D)$ is placed on functional unit $j \in V(G)$.
$r_{i,j}$	Binary variable: whether dependency $i \in E(D)$ is maintained via routing nodes $j \in V(G)$.
Res_i	The set of routing nodes which abstract link i covers.
$cr_{i,j}$	Binary variable: whether dependency $i \in E(D)$ is maintained via abstract link j .
Fus_i	The set of FUs which hardware cluster i covers.
$cf_{i,j}$	Binary variable: whether operator $i \in V(D)$ is placed on hardware cluster j .
$HG(V, E)$	$V(HG)$ is the set of hardware clusters and $E(HG)$ is the set of abstract link between them.
$ce_{i,j}$	Binary variable: whether dependency $i \in E(D)$ is maintained via $j \in E(HG)$.
$FD(V, E)$	The graph generated from DFG by fusing operators.
Ops_i	The set of operations which software cluster i covers.
ST	$\forall X \in ST, \forall u, v \in X, u$ and v are clustered together by temporal clustering.
t_i	The control step on which operator $i \in V(D)$ is scheduled.
SP	$\forall \{u, v\} \in SP, u$ and v are clustered together by throughput-sensitive clustering.
HD_{hc_i, hc_j}	$\forall hc_i, hc_j \in V(HG), HD_{hc_i, hc_j}$ is the total number of the path from hc_i to hc_j .
HFO_{hc_i, hc_j}	$\forall hc_i, hc_j \in V(HG), HFO_{hc_i, hc_j}$ is the maximum fan-out from one FU in hc_i to any FU in hc_j .
HF_{hc_i, hc_j}	$\forall hc_i, hc_j \in V(HG), HF_{hc_i, hc_j}$ is the maximum fan-in from one FU in hc_i to any FU in hc_j .
$cb_{op, hc}$	Binary variable: whether fused operator $op \in V(FD)$ is mapped onto hardware cluster $hc \in V(HG)$.
$eb_{se, hc}$	Binary variable: whether dependency $e \in E(FD)$ is mapped onto a data path $hc \in E(HG)$.
OpT	Operator type set of the original DFG.

be reconfigured at runtime. The CU is responsible for providing communication between producer and consumer FUs to sustain the data dependency (e.g., multiplexer, register). For the purpose of clustering hardware elements described in low-level hardware IR, CaSMap takes a two-step strategy, including CU Clustering and FU Clustering:

Connection Unit Clustering: CU Clustering aims to eliminate redundancy and allows for a simplified representation of low-level hardware IRs, so that the scale of the mapping model shrinks. There are four kinds of interconnection patterns existing in the current RSA design: mesh-like, crossbar, point-to-point, and bus. We focus on the most popular mesh-like RSAs (e.g., [33, 38, 47]) and crossbar-based (X-bar) RSAs (e.g., [14, 20, 43, 54, 56]) because of their good

scalability and high energy efficiency [58]. It is noteworthy that mesh-plus, depopulated crossbar/mesh, hierarchical interconnections are all supported because CUs are clustered according to distances and other structure-neutral metrics. Sec. 3.6 will discuss the other interconnection types.

Mesh-like RSAs provide specific data paths for each pair of neighbor FUs. More concretely, the occupation of the routing nodes on the data path can be settled once the producer and consumer FUs in neighborhood are given. As a result, it is unnecessary to introduce superfluous variables to describe data propagation details between FUs. In this manner, the routing nodes on the path from one FU to its neighbor FU can be clustered into one or more abstract links. One abstract link is enough when the source FU's output can be reached by any input of the destination FU.

X-Bar RSAs take advantage of a network of crossbar switches as the foundation of communication between FUs. A crossbar switch, which can be implemented with multiple multiplexers, is designed to provide connectivity from each of its inputs to each of its outputs. Therefore, the crossbar switch network supplies greater interconnection flexibility which enables single-cycle multi-hop reachability. Nevertheless, no switch is used exclusively by communication of any pair of FUs. Therefore, crossbar unit should not be clustered directly with other ordinary CUs. Figs. 2b and 2d shows the CU clustering results when targeting the two kinds of RSA topologies (MorphoSys and HyCUBE as representatives).

Functional Unit Clustering: For the purpose of clustering FUs, we propose a novel communication distance model based on the connection topology. We define the communication distance (CD) between each pair of FUs firstly and then aggregate the adjacent FUs together to form a high-level hardware IR because communication is more likely and efficient to occur among adjacent FUs.

The CD from FU u to FU v is defined as the minimum *delay* of all reachable paths, as shown in Eq. 1 where $V(H)$ denotes the FU set of RSA H . For mesh-like RSAs, the *delay* of a path is defined as the number of delay units (registers) passed through. For X-Bar RSAs, because the circuit critical path delay is largely affected by the allowable maximum hops within one clock cycle, the *delay* calculation should also include the impact of crossbars. Figs. 2b and 2d show the representative examples for CD calculation. In Fig. 2b, the CD between FU0 and FU1 equals to 1 because there is at least one register except combinational logic on the data path between them. In Fig. 2d, we endow each crossbar switch with *delay* contribution of 0.2, and the CD between FU2 and FU4 is equal to 1.6 because of crossing at least one register and three crossbar switches.

$$\forall u, v \in V(H), CD(u, v) = \min_{p=u \leadsto v} \text{delay}(p) \quad (1)$$

Given a communication distance threshold $CDTH$ to restrict the upper bound communication distance of two FUs belonging to one cluster, FU clustering scheme can be converted to a graph coloring problem by constructing an intermediate graph (IG) where each vertex represents an FU as in Eq. 2. According to the graph coloring result of IG, FUs with the same color are clustered together to form a composite vertex (hardware cluster) in high-level hardware IR as shown in Figs. 2f and 2h (each crossbar switch is distributed to the same hardware cluster as its nearest FU).

$$\forall u, v \in V(H), IG(u, v) = \begin{cases} 0, & CD(u, v) \leq CDTH \\ 1, & CD(u, v) > CDTH \end{cases} \quad (2)$$

The hardware clustering algorithm is presented in Algorithm 1. First, Graph H is generated by CU clustering and setting proper delays (w) to the connection units (line 1). Next, Dijkstra's shortest path algorithm is called to calculate the minimum delay between any pair of FUs and obtain the CD matrix (lines 2-3). Then, we construct and modify the intermediate graph (line 4-9), and use a graph coloring algorithm to obtain the final clustering result (line 10).

Hardware Clustering Constraints: The CU clustering adds a set of variables $cr_{i,j}$ that constrain the original variables that define routing: $\forall i \in E(D), \forall \text{abstract link } j, \forall k \in Res_j, r_{i,k} \geq cr_{i,j}$. The FU clustering also adds new variables $cf_{i,j}$ and $ce_{i,j}$ that constrain the original variables defining placement and routing: 1) $\forall i \in V(D), \forall j \in V(HG), \sum_{k \in Fus_j} f_{i,k} = cf_{i,j}$. 2) $\forall i \in E(D), \forall j \in E(HG), \forall \text{abstract link } k, k.src \in j.src, k.dst \in j.dst, \sum_k cr_{i,k} = ce_{i,j}$.

There are two purposes of hardware clustering. Firstly, it can reduce the scale of the mapping problem. The interconnection among FUs in the same hardware cluster is relatively flexible (even fully-connected). Therefore, in most situations, we only need to care about the placement and routing between clusters, because dependencies can be easily maintained within one cluster. In addition, hardware clustering is able to provide heuristic for high-quality mapping. The communication delay among FUs in the same hardware cluster is relatively low. Therefore, encouraging intra-cluster communication probably results in a mapping with high performance.

Algorithm 1: Hardware clustering algorithm (HW_Cluster).

Input: $MRRG, CDTH, w$
Output: HWC

```

1  $H \leftarrow CUCluster(MRRG, w);$ 
2 for  $u \in V(H)$  do
3    $CD[u, :] \leftarrow Dijkstra(H, u);$ 
4  $IG.Initialize(V = V(H), E = \emptyset);$  //Intermediate Graph
5 for  $u \in V(H)$  do
6   for  $v \in V(H)$  do
7     if  $CD(u, v) > CDTH$  then
8        $IG.addEdge(u, v);$ 
9  $IG.RemoveEdgeDirection();$ 
10  $HWC = GraphColoring(IG);$ 

```

3.3 Software Clustering

Since over 90 percent of the total execution time of data- and compute-intensive applications like digital signal processing and cryptographic algorithm is spent in loops [28], we focus on the mapping problem of loop kernels. To illustrate the process of software clustering, we adopt DFG to express software IR. In this section, we introduce three optional strategies for software clustering.

Operator Fusion: Operators can be fused by analyzing the paradigm of domain-specific language (DSL) or domain-specific pattern. In a specific domain, a combination of frequent operators which dominates the execution time of algorithms is the bottleneck (e.g., multiply accumulate (MAC)). To eliminate domain-specific bottlenecks, many RSAs provide dedicated hardware (usually organized as processing elements) for these operators. Therefore, a group of relative operators that can be accelerated by dedicated resources tend to be clustered together for higher performance.

Throughput-Sensitive Clustering: The objective of loop mapping is to minimize the initiation interval (II), the interval between two consecutive iterations of a given loop, which has a direct impact on the final throughput [3, 10, 64]. A smaller II indicates a shorter execution time per loop, and therefore a higher throughput rate. The theoretical lower bound of II, termed MII, equals to the larger of RecMII and ResMII, where ResMII is the minimum II constrained by resources and RecMII is derived from inter-loop dependencies [44]. The software clustering purposes to provide heuristic for the following mapping process, and exerts influence on the convergence speed and throughput. Therefore, it should follow two rules: a) A pair of operators, which are connected with a throughput-sensitive dependency, ought to be clustered together. b) Edges with long latency is preferred to be placed across clusters. The above two rules essentially take account of recurrence and resource factors that may affect the throughput. Firstly, the communication represented by a throughput-sensitive edge should be low-cost in order to avoid further enlargement of RecMII. In addition, because edges with longer latency require more resources to buffer the intermediate results, to allocate more hardware to share the resource burden is beneficial to suppressing the expansion of ResMII.

Fig. 3 depicts examples of software clustering. Based on the multi-level mapping model which will be introduced later, operators belonging to one software cluster are prone to be placed onto FUs in the same hardware cluster. Taking the architecture in Fig. 2e as target and adopting the hardware clustering pattern shown in Fig. 2f, we get the final mapping results. As shown in Fig. 3a, e_1 and e_2 form a ring and thus are throughput-sensitive, and e_3, e_4 ,

and e_5 are dependencies with long latency after dataflow analysis. Therefore, the clustering result is shown in Fig. 3a. Fig. 3c shows an arbitrary clustering result which cuts e_1 and e_2 and reserve e_3 ,

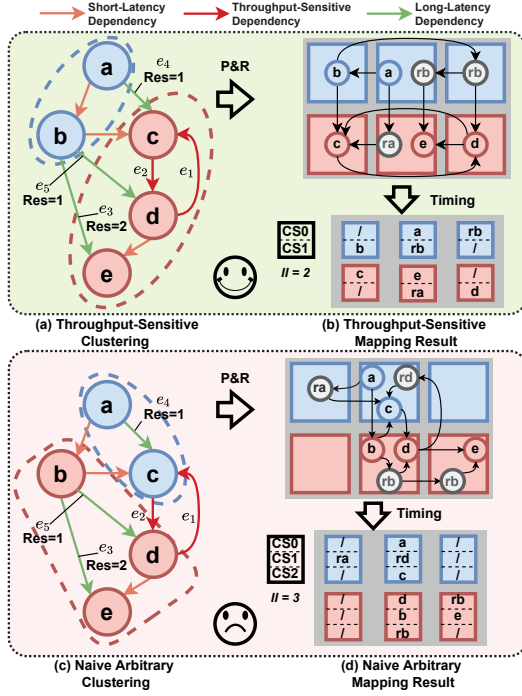


Figure 3: Software clustering strategy will impact the quality of the final mapping result. Throughput-sensitive clustering (a) is able to achieve a better mapping than arbitrary clustering (c).

Temporal Clustering: By analyzing the dependencies in DFG, each operator can be assigned a time label. Therefore, operators with the same time label is probably executed at the same control step. Modulo scheduling [44], an approach to realizing software pipelining, which aggressively exploit parallelism by scheduling operations from several loop iterations to the same time slot, can be also treated as a kind of this clustering strategy.

Software Clustering Constraints: Operator fusion bundles the original variables that define placement with new variables $cb_{i,j}$: $\forall j \in V(HG), \forall i \in V(FD), \forall k \in Ops_i, cf_{k,j} = cb_{i,j}$. They also bundles the original variables that define routing with a set of new variables $eb_{i,j}$: $\forall i \in E(FD), \forall j \in E(HG), \forall k \in E(D), k.src \in i.src, k.dst \in i.dst, ce_{k,j} = eb_{i,j}$. Throughput-sensitive clustering bundles some of the variables: $\forall \{i, j\} \in SP, \forall k \in V(HG), cb_{i,k} = cb_{i,j}$. Temporal clustering fixes the timing sequence by adding constraints: $\forall X \in ST, \forall u, v \in X, t_u = t_v$.

3.4 Multi-Level Mapping Model

CaSMap framework is able to automatically establish a multi-level mathematical model which describes the mapping problem based on software and hardware clustering. By decomposing and scattering the integrated mapping model into a high-level model plus one or more low-level models, the original mapping problem can be solved agilely.

High-Level Mapping Model: As mentioned in the previous section, before setting up the mapping model, clustering algorithms analyze the software and hardware IRs and obtain some heuristics in the form of new variables and constraints related to high-level IRs. In order to carry out these heuristics, CaSMap framework firstly attempts to find a mapping between clustered software and hardware IRs with constrained optimization approaches. The high-level model has two characteristics: relatively small scale and high success rate for the following low-level mapping. We use linear constraints and objectives to establish mapping models in the form of ILP as follows. The constraints and objectives are derived from the original mapping model and the hardware/software clustering constraints brought by the organization of multi-level IRs.

Operator Placement Constraint: Every fused operator in FD is placed on exactly one hardware cluster which is functionally compatible (Eq. 3).

$$\forall op \in V(FD), \sum_{hc \in V(HG), Func(op) \subseteq Func(hc)} cb_{op,hc} = 1 \quad (3)$$

Throughput-Sensitive Constraint: Each pair of operators in SP should be placed onto the same hardware cluster (Eq. 4).

$$\forall \{op_0, op_1\} \in SP, \forall hc \in V(HG), cb_{op_0,hc} = cb_{op_1,hc} \quad (4)$$

Resource Constraints: Each type of hardware's compute resource is finite on each control step (Eq. 5).

$$\begin{aligned} \forall X \in ST, \forall p \in OpT, \forall hc \in V(HG), \\ \sum_{op \in X} size_p(op) \times cb_{op,hc} \leq size_p(hc) \end{aligned} \quad (5)$$

Communication Constraints: Every edge in fused DFG (FD) occupies at least one path between hardware clusters or inside one cluster (Eq. 6). The bandwidth of a path inside one hardware cluster or across two clusters is finite (Eq. 7). If two operators communicate through a path he , their placement are also determined (Eq. 8).

$$\forall se \in E(FD), \sum_{hc \in E(HG)} eb_{se,hc} \geq 1 \quad (6)$$

$$\begin{aligned} \forall X \in ST, \forall he = (he_s, he_d) \in E(HG), \\ \sum_{se=(op_s, op_d) \in E(FD), op_d \in X} eb_{se,he} \leq HD_{he_s, he_d} \end{aligned} \quad (7)$$

$$\begin{aligned} \forall se = (op_s, op_d) \in E(FD), \forall he = (he_s, he_d) \in E(HG), \\ cb_{op_s, he_s} + cb_{op_d, he_d} \geq 2e_{se, he} \end{aligned} \quad (8)$$

Fan-in and Fan-out Constraints: Communications between fused operators cannot exceed the maximum fan-in or fan-out (Eq. 9 and Eq. 10).

$$\begin{aligned} \forall op \in V(FD), \forall he = (he_s, he_d) \in E(HG), \\ \sum_{se=(op_s, op_d) \in E(FD), op_d=op} e_{se, he} \leq HFI_{he_s, he_d} \end{aligned} \quad (9)$$

$$\forall op \in V(FD), \forall he = (he_s, he_d) \in E(HG), \quad (10)$$

$$\sum_{se=(op_s, op_d) \in E(FD), op_s=op} e_{se,he} \leq HFO_{he_s,he_d}$$

Initiation Interval Objective: We always pursue a minimal Π for the purpose of higher performance (Eq. 11). The more evenly the operators are distributed to the FUs, the better the performance is.

$$\text{Minimize} \max_{hc \in V(HG), p \in Opt} \frac{\sum_{op \in V(FD)} size_p(op) \times cb_{op,hc}}{size_p(hc)} \quad (11)$$

Communication-Sensitive Objective: As mentioned in Section 3.2, the connection flexibility of FUs within the same hardware cluster is much greater than FUs from different clusters. Communication cost between two FUs within the same hardware cluster is smaller as well. Therefore, minimizing cross-cluster communications is advantageous to both mapping success rate and quality. The expression is shown in Eq. 12.

$$\text{Minimize} \sum_{se \in E(FD)} \sum_{he \in E(HG), he.src \neq he.dst} e_{se,he} \quad (12)$$

Low-Level Mapping Model: After conveying the solution of the high-level mapping model to the low-level mapping model, the scale of the low-level model shrinks because a lot of variables are deteriorated to constants through the software and hardware clustering constraint propagation. In some cases, the low-level model can be divided into several independent sub-models to be solved more quickly. The division process can be done by the state-of-the-art solver [1]. Therefore, the scattered mapping process is capable of finding a favorable mapping result more quickly. The low-level model can be set up on the basis of previous researches [8, 41] with low-level IRs by adding an interface to the high-level mapping model.

3.5 Automatic Mapping Iteration

It is inevitable that the multi-level model could make the original solvable problem fail to be solved because of inappropriate clustering strategies or high-level mapping result. Therefore, CaSMap enables iterative execution with constraint reduction and conversion, as shown in Fig. 1, to deal with the mapping failure.

As mentioned in Sec. 3.2 and Sec. 3.3, IR clustering and high-level mapping has the same effect as bundling variables (i.e., constraining a bunch of variables to take values in the same determinate domain). Therefore, it may result in failure when some of these constraints are incompatible. CaSMap is able to automatically delete these constraints or convert them into objectives when meeting failures. Finally, the mapping model evolves to become suitable for the mapping problem.

The constraint reduction and conversion sequence is based on the predefined reduction priority. Among the proposed constraints in this work, the reduction priorities can be defined as follows (from high to low): throughput-sensitive clustering, operator fusion, temporal clustering, FU clustering, and CU clustering. That is because the first two constraints are more likely to bring functional incompatibility to make the high-level mapping fail and throughput-sensitive clustering contributes to the mapping quality more than feasibility. Temporal clustering (for static-scheduling RSAs) may be inappropriate because of ignoring the hardware related constraints. CU clustering has the lowest reduction priority, because it only eliminates representation redundancy of low-level hardware IRs.

Fig. 4 depicts two examples of CaSMap process. Figs. 4a-4d illustrate the process of CaSMap when mapping the DFG in Fig. 4a onto a static-scheduling RSA with mesh-like interconnection. Fig. 4a is the multilevel-level software IR after operator fusion, throughput-sensitive clustering, and temporal clustering. As shown in Fig. 4a, fused operators in the same shape are clustered together to the same control step and operators d and g are supposed to be placed on the same hardware cluster for high throughput. The high-level mapping result is obtained in Fig. 4c where operators a , b , d , and g are placed on *Hardware Cluster 0 (HW0)* and the other operators are placed on *Hardware Cluster 1 (HW1)*. Because PEs in the same hardware cluster are fully-connected, only caring about the communications across hardware clusters (edges (a, c) , (b, e) , and (g, h)) is enough. After the solving the low-level model, the final mapping result can be obtained (Fig. 4d). Lower-level mapping details are not illustrated because they do not help in visualization.

Figs. 4e-4g illustrate the process of CaSMap when mapping the DFG in Fig. 4e onto a dynamic-scheduling RSA in Fig. 4f. Initially, the software and hardware IRs are clustered with the proposed approaches. The communication distance threshold (CDTH) for hardware clustering equals to 1. The high-level mapping fails because there is not a hardware cluster that supports both MUL and ADD. Next, high-level mapping executes again after software clustering constraints are converted to objectives. Unfortunately, the following low-level mapping fails because dependency ($MUL4, ADD3$) cannot be satisfied. Then, the FU clustering constraints are eliminated, and therefore high-level mapping conveys no partition to the low-level mapping model. Finally, CaSMap finds a valid mapping through the low-level mapping process.

3.6 Generality Discussion

CaSMap can be applied to dynamic scheduling reconfigurable spatial architectures. To orchestrate functional units and connection units in static-scheduling RSAs, the mapper must determine their execution sequences and functions in advance. Nevertheless, scheduling can be neglected by the mapper when handling dynamic scheduling RSAs, because the execution sequence is determined by the dynamic scheduling logic on the fly. Therefore, the problems of orchestrating dynamic scheduling functional and connection units are only a subset of the problems of orchestrating static-scheduling ones. Constraints related to scheduling (Eqs. 5&7) can be omitted when dealing with dynamic scheduling RSAs.

CaSMap is able to handle heterogeneous architectures and irregular interconnections. Heterogeneous architectures possibly make the high-level mapping process more likely to fail due to functional inconsistency between software clusters and hardware clusters, as shown in Fig. 4e and Fig. 4f. CaSMap executes iteratively with constraint reduction and conversion introduced in Sec. 3.5 to handle this scenario. Similarly, CaSMap can also deal with irregular interconnections. Moreover, point-to-point and bus interconnections can be treated as fully-connected mesh-like network and simplify the mapping problem a lot. As RSAs are becoming more area/energy efficient as domain-specific accelerators, these two kinds of interconnections are decreasing.

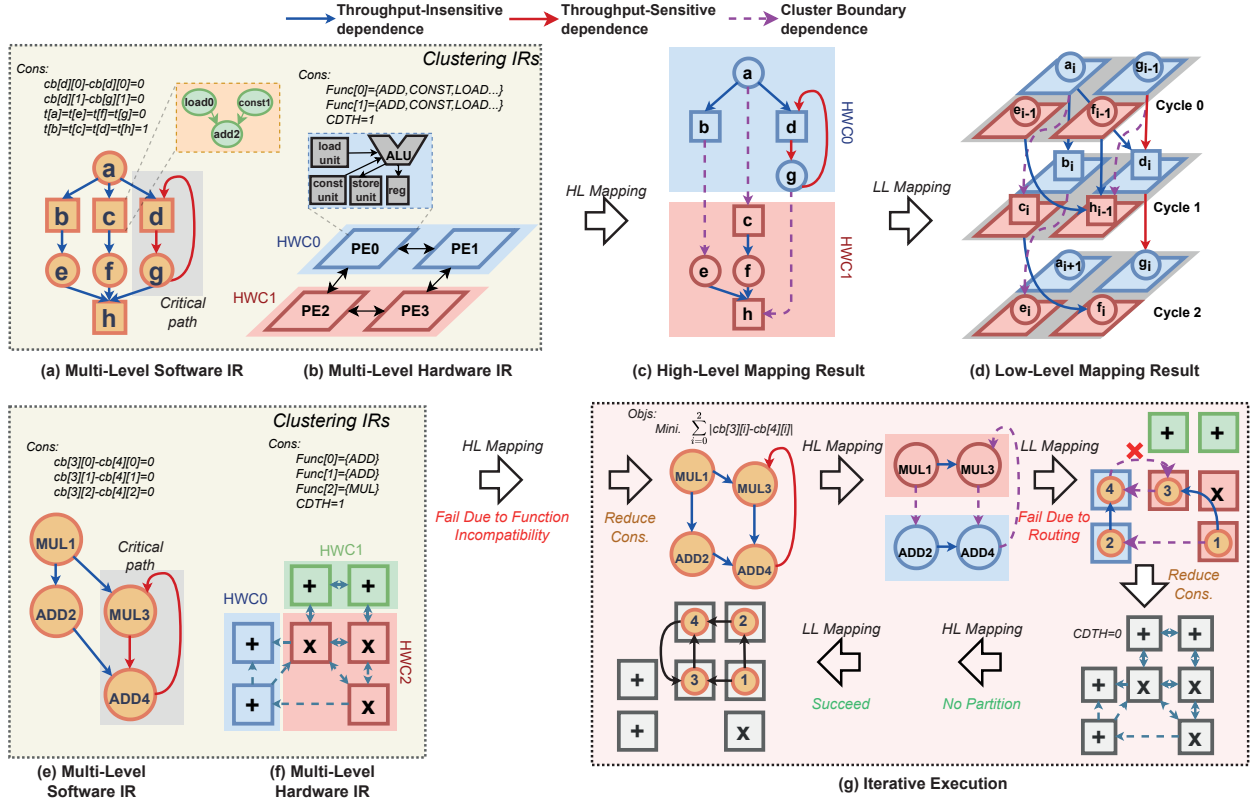


Figure 4: Two examples of CaSMap process when handling different applications and architectures. (a)-(d) show an example without mapping failures. (e)-(g) illustrate the iterative execution when meeting failures.

4 EVALUATION

In this section, we demonstrate the model scale, time cost, mapping quality compared with a waterfall ILP mapper and a simulated annealing based mapper.

4.1 Experimental Setup

Input Software and Hardware: To test the proposed CaSMap framework, we select a set of loop kernel benchmarks as input software and four RSAs as input hardware. The benchmarks come from Microbench [9], ExPRESS benchmark [21], and Embench-IoT [5]. The target architectures include MorphoSys [47], HReA [33], HyCUBE [20], and REVEL [56]. MorphoSys and HReA adopt chordal ring interconnection topology with degrees of 6 and 10 respectively. To help visualize the interconnection, only one PE's neighborhood is illustrated in Figs. 5a and 5b. HyCUBE and REVEL are constructed with crossbar-based (X-bar) interconnection as shown in Fig. 5c and 5d. Because each of these architectures, except the general purpose HReA, is designed for a specific application domain, they differ a lot in PE organization. For example, REVEL is a hybrid architecture with static- and dynamic-scheduled PEs. For all the benchmark loop kernels, hybrid architecture is meaningless. Therefore, we substitute the static-scheduling PE of HReA for all the PEs of the other three architectures, in order to support the selected loop kernels. Also, we assume there is only one local register that

acts as the output register in each PE. We normalize the PE array to the same size of 4x4 for model scale and time cost comparisons in Sec. 4.2 and Sec. 4.3.

Low-Level Hardware and Software IRs: We take the modulo routing resource graph (MRRG) [36] as the low-level hardware IR and it can be easily transformed from an XML-based hardware construction language, CGRA-ADL [9]. We model the PE with five constant units, two load units, one ALU, one store unit, and one register. We take the DFG as the low-level software IR which can be transformed from the LLVM IR. The DFGs are generated after optimization for the target architecture (DFG balancing and memory access alignment operation elimination). Table 4 shows the features of the selected kernels' DFGs.

Baseline Mapper: We take the waterfall ILP mapper of CGRA-ME, AA-ILP, as the first baseline. It directly sets up and endeavors to solve an integrated ILP model which describes the whole mapping problem. All the constraints and objectives can be found in [9]. To compare CaSMap with heuristic methods, we take the simulated annealing (SA) based mapper, DRESC [36], as the second baseline. **Modeling Method and Solver:** For fair comparison, we also adopt ILP modeling in the scattered mapping process. All the ILP models of AA-ILP and CaSMap are solved by Gurobi 9.0.0 [42], a commercial optimization solver for ILP. Gurobi can use multiple cores of a machine to speed up the process of solving ILP problems. All

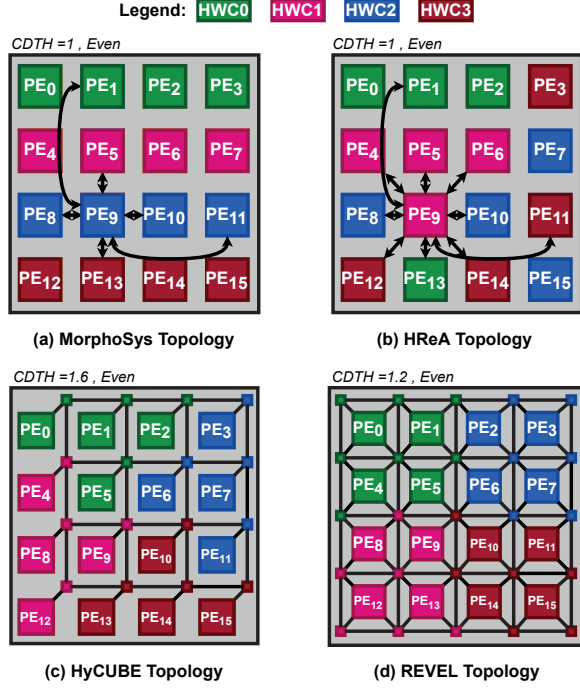


Figure 5: The target architectures and hardware IR clustering examples.

Table 4: DFG features of the selected loop kernels.

Benchmarks	Vertices	Edges
accum	21	25
cap	42	47
conv2	18	20
conv3	28	31
mac	12	14
mac2	40	46
matmult	26	28
mults1	34	38
mults2	42	48
sum	8	9
arf	54	86
h2v2	68	71
mulul	97	108

experiments are carried out on the same machine with 4 Intel(R) Core(TM) i5-4690 CPUs @ 3.50GHz.

4.2 Mapping Model Scale

An ILP problem can be converted to the standard expression in Eq. 13 after presolve. We define the model scale as the number of nonzero elements in coefficient matrix A .

$$\begin{aligned} & \text{maximize } \mathbf{c}^T \mathbf{x} \\ & \text{subject to } \mathbf{A}\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}, \mathbf{x} \in \mathbb{Z}^n \end{aligned} \quad (13)$$

Fig. 6 Cols. 1-3 shows the detailed ILP scales of AA-ILP and CaSMap with different software and hardware IR clustering strategies across different RSAs. The total ILP scale of CaSMap (Col. 1) is obtained by summation of the high- and low-level model scales (Cols. 2&3). For each RSA, we test three kinds of IR clustering strategies: one with only software IR clustering (SC) and another two

with software IR clustering combined with different hardware IR clustering methods (SCHC x). For mesh-like architectures, x means the communication distance threshold (CDTH) which is early defined in Section 3.2. Because the maximum CD of every pair of PEs in HReA is equal to 2, the clustering algorithm will cluster all PEs together and obtain a meaningless clustering result, when assigning 2 to CDTH. Therefore, we adopt two kinds of hardware IR clustering results for experiments with the CDTH of 1: even and uneven (i.e., all PEs are evenly distributed to clusters and not), denoted by SCHC1 and SCHC1_1 respectively. For X-Bar architectures, x means the maximum crossbar switches, across which each pair of PEs belonging to the same hardware cluster can reach each other. One clustering example for each RSA is shown in Figs. 4a-4d (assuming a crossbar unit's delay contribution to CD equals 0.2). We can find that the scale of low-level models can be reduced with the help of the multi-level decomposition. Experimental results show that only with software IR clustering, CaSMap can reduce 46.0% (MorphoSys), 44.9% (HReA), 7.5% (HyCUBE), and 10.9% (REVEL) scale of the mapping model on average compared with AA-ILP respectively. Combined with hardware IR clustering, the geo-mean scale reductions become up to 93.5% (MorphoSys), 94.9% (HReA), 23.1% (HyCUBE), and 42.1% (REVEL). The scales of mapping models for X-Bar architectures do not shrink as much as those of mesh-like architectures due to the fact that great interconnection flexibility brought by ample shared crossbar switches needs more variables to express.

4.3 Time Cost

In both AA-ILP and CaSMap framework, the time cost of solving ILP models dominates the total execution time. We take Gurobi runtime as the time cost metric. The objective of the mapping model is to pursue a minimal II and minimize the occupation of routing resources: $\sum_{i \in E(D)} \sum_{j \in V(G)} r_{i,j}$, just as Eq. 10 in [8]. To minimize the II, both AA-ILP and CaSMap start with the target II equal to MII and increase the target II when meeting failure. To minimize the routing resource occupation, we set the AA-ILP and CaSMap's low level mapping model equal to the routing resource occupation with relative optimality gap equal to 20% and 0 for mapping models of architectures with X-Bar and mesh-like interconnection respectively because of the difference in problem scale. We set the mapping time limit to 8 hours.

Comparison with Waterfall ILP Mapper: By comparing the total Gurobi runtime of CaSMap and AA-ILP (WILP) in Fig. 6 Cols. 4-6, we can find the time cost has a similar trend with the model scale in Fig. 6. Experimental results show that all mappings using CaSMap can be finished within the time limit. The average time cost of CaSMap (with clustering strategy SCHC3 for HyCUBE topology and SCHC1 for the others) is 99.8 (0.02-3263.4) seconds. The average time cost of AA-ILP is 8348.3 (0.10-28800) seconds (12 cases time out). The mapping time achieves a speedup of 83 $\times$ over AA-ILP. The average time costs of software/hardware IR clustering are 7.6 (6.2-8.9) ms and 74 (36-96) ms. The total clustering time cost is less than 0.1% relative to the time cost of the core ILP solving time.

Comparison with Simulated Annealing Based Mapper: We compare the time costs of simulated annealing based DRESC and CaSMap (with clustering strategy SCHC3 for HyCUBE topology

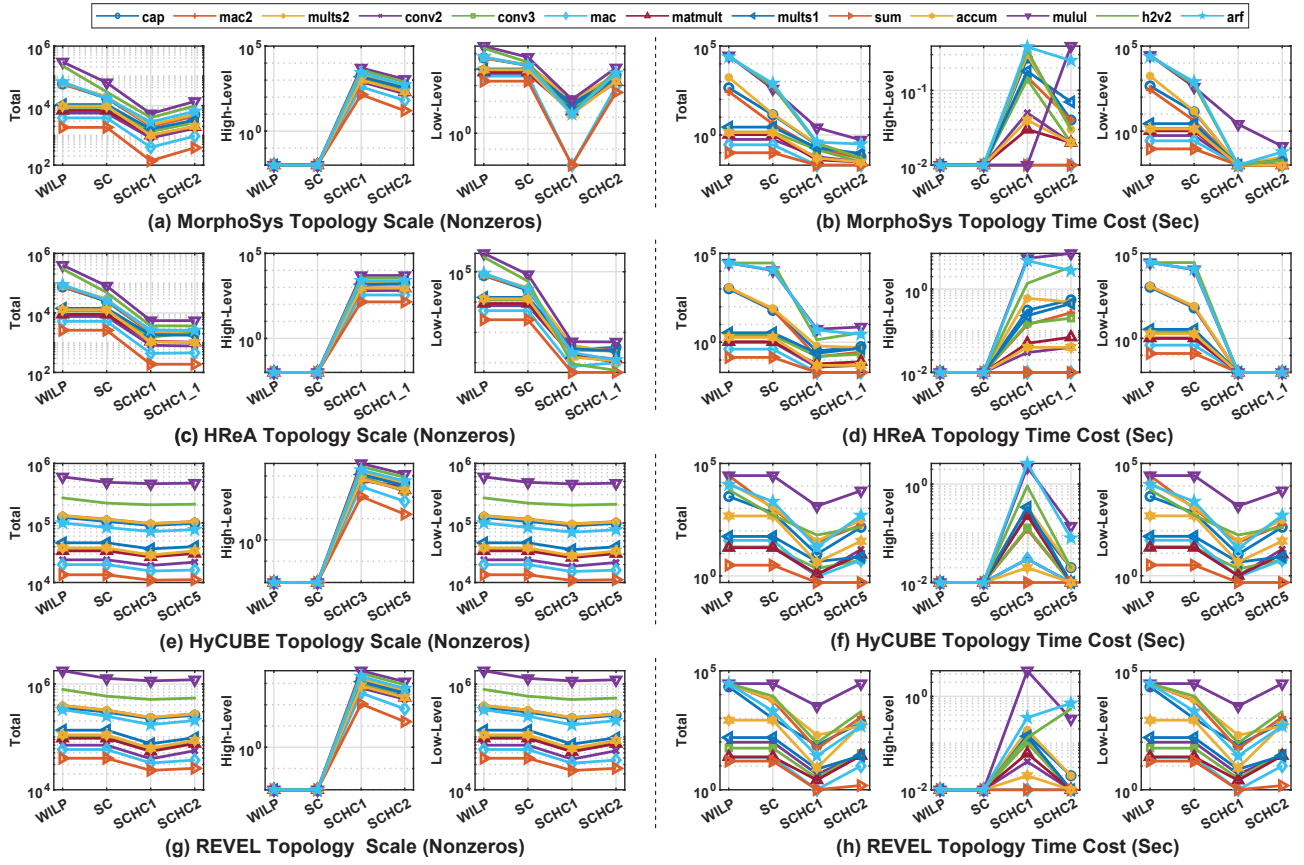


Figure 6: The ILP scale and time to solution decomposition of AA-ILP (WILP) and CaSMap with different IR clustering strategies in CaSMap compared with AA-ILP across four different RSA topologies (The model scale of 0 is denoted by 10^{-2} for illustration on the logarithmic axis).

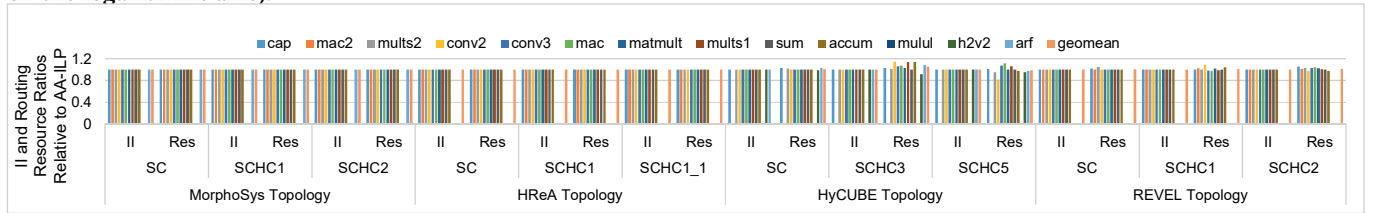


Figure 7: The II and routing resource occupation comparisons between AA-ILP and CaSMap with different IR clustering strategies across four different RSA topologies.

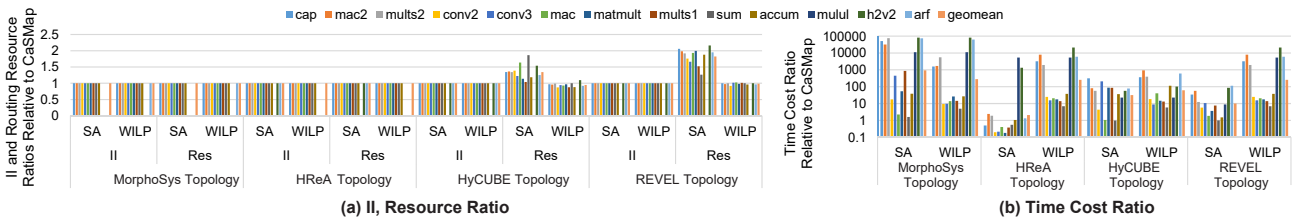


Figure 8: The II, routing resource occupation, and time cost comparisons between simulated annealing based mapper, DRESC (SA), waterfall ILP mapper, AA-ILP (WILP), and CaSMap. SA is not suitable for MorphoSys and slower than AA-ILP because of its sparse inflexible mesh-like interconnection.

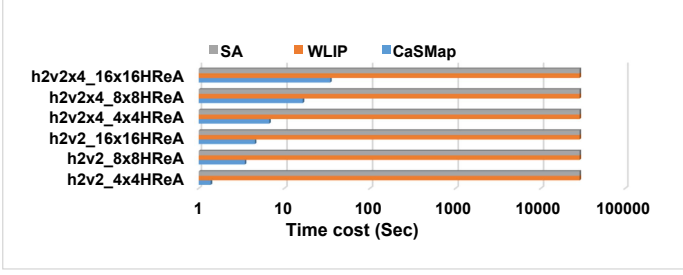


Figure 9: The time costs of mapping an unrolled DFG (h2v2x4, 272 vertices and 284 edges) using CaSMap onto RSAs with different scales.

and SCHC1 for the others), as shown in Fig. 8b. CaSMap achieves 2x, 32x, and 10x geo-mean speedup over the SA mapper for HReA, HyCUBE, and REVEL topologies respectively. The comparisons of the two optimization objectives (II and routing resource occupation) will be discussed in Sec. 4.4.

Scalability: We unroll *h2v2* four times (h2v2x4 in Fig. 9) and test the time costs of mapping it on to 4x4, 8x8, and 16x16 RSAs with HReA-like interconnection respectively. CaSMap’s results show good scalability, while AA-ILP and DRESC time out in all cases shown in Fig. 9.

Table 5: The success rate of CaSMap without Eq. 12 and routing resource occupation variations without Eq. 12.

RSAs	MorphoSys		HReA		HyCUBE		REVEL	
CaSMap Strategy	SCHC1		SCHC1		SCHC3		SCHC1	
Benchmarks	Mapped	Res Ratio	Mapped	Res Ratio	Mapped	Res Ratio	Mapped	Res Ratio
cap	0	-	1	100.0%	1	102.8%	1	112.5%
mac2	0	-	1	100.0%	1	102.0%	1	104.2%
mults2	0	-	1	100.0%	1	106.5%	1	102.1%
conv2	0	-	1	100.0%	1	100.0%	1	95.3%
conv3	0	-	1	100.0%	1	111.1%	1	108.0%
mac	0	-	1	100.0%	1	96.4%	1	104.1%
matmult	0	-	0	-	1	106.3%	1	99.6%
mults1	0	-	1	100.0%	1	99.0%	1	111.3%
sum	1	100.0%	1	100.0%	1	100.0%	1	100.0%
accum	0	-	1	100.0%	1	88.1%	1	104.6%
mulul	0	-	1	100.0%	1	115.8%	1	81.6%
h2v2	0	-	1	100.0%	1	123.3%	1	105.6%
arf	0	-	0	-	1	98.2%	1	112.7%
Sum/Geomean	1	100%	11	100%	13	103.5%	13	103.6%

4.4 Mapping Quality and First-Try Success Rate

The ratios of II and routing resource occupation of CaSMap relative to AA-ILP is shown in Fig. 7. CaSMap is able to reach identical optimal IIs to AA-ILP in all the mappings that do not time out using AA-ILP. In theory, AA-ILP can, given enough time, achieve the optimal II because it models the complete solution space of the mapping. For mesh-like RSAs, the routing resource occupations are also identical to those of AA-ILP due to interconnection exclusivity. For X-Bar RSAs, because we set the relative optimality gap to 20%, the routing resource occupation of AA-ILP is not always optimal and CaSMap occupies 5.5% and 1.2% more routing resources on average compared with AA-ILP respectively. As shown in Fig. 8a, CaSMap occupies 25.5% and 45.1% less routing resource on average than SA-based mapper, DRESC, on HyCUBE and REVEL topologies respectively.

As mentioned in Section 3.2, Eq. 12 aims to increase the first-try success rate and reduce routing resource occupation. To demonstrate the effectiveness of this objective we select a clustering strategy for each RSA and list the mapping results (success or failure) and routing resource occupation ratios without Eq. 12 relative to those with Eq. 12 in Table 5. For mesh-like RSAs, the first-try success rate of the mapping process becomes 46.1% (with Eq. 12, the success rate is 100%). That is because mesh-like RSA cannot handle too many communications across clusters due to its poor flexibility of interconnection (especially for MorphoSys). For X-Bar RSAs, although the success rates remain 100%, the resource occupation increases 3.5% on average. The great flexibility provided by cross-bar switches gives more chances for successful mappings, and it is more difficult to find an efficient mapping rather than a valid one. Because Eq. 12 encourages PEs to communicate within a hardware cluster, the routing resource occupation can be reduced.

4.5 Heterogeneous Architecture

To demonstrate the generality of CaSMap, we also test CaSMap on a heterogeneous RSA with hybrid interconnection as shown in Fig. 10a. An RPE in Fig. 10a represents a reduced PE without multiplier. The experimental results are shown in Fig. 10b. Compared with AA-ILP, CaSMap can find a mapping result with only 6.1% more routing resource occupation and the same II, and achieve a mapping time speedup of 4.5x, with no more than two iterations.

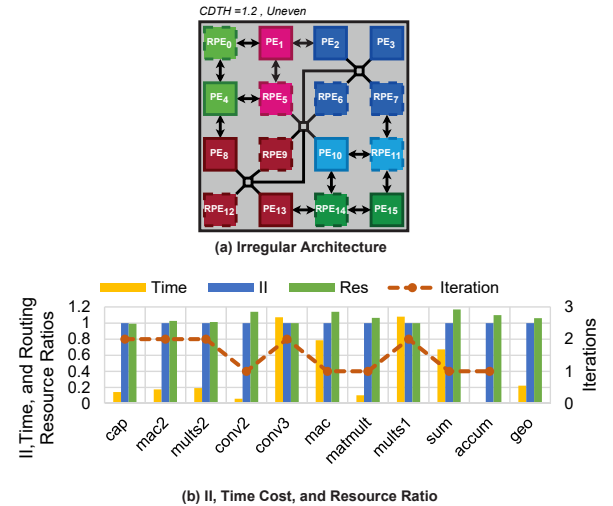


Figure 10: The II, time cost, routing resource occupation ratios relative to AA-ILP on an irregular RSA.

4.6 More Discussion

Hardware clustering strategy has an impact on the mapping efficiency, as shown in Fig. 6 and Fig. 7. For example, the mapping time of SCHC1 is 8.3x and 27.3x faster than that of SCHC2 and SC on REVEL respectively, while their IIs are equal and resource occupations are close. Supposing that a crossbar unit’s delay contribution to communication distance (CD) equals 0.2, these three

clustering strategies are equal to sweeping the communication distance threshold (CDTH) with the value of 0 (SC), 1.2 (SCHC1) and 1.4 (SCHC2). The reason is that when CDTH equals to 1.2, any pair of PEs from the same hardware cluster can communicate with each other at the same time without conflicts. It can prune a lot of searching space for the low-level mapping while the scale of high-level mapping model is not huge at the same time. CDTH is not an input hyperparameter because it is related to interconnection density and can be obtained from multiple iterations. The other clustering strategies also have no hyperparameters.

5 RELATED WORK

Mapper and Scheduler: The integrated mappers solve the scheduling, placement, and routing problems in one shot with constraint-based model [8, 11, 37, 41] or heuristic algorithms [13, 36, 50]. They either struggle with unacceptable time cost faced with large problems or needs expertise of specific architectures. Decomposed mappers [10, 23, 52, 64] split the original problem into smaller subproblems and solve them step-by-step. It is prone to prevent certain mappings from being possible due to restrictions from earlier phase result, and thus the solution quality may be low. Scheduler for RAW architectures [19] considers both ILP and heuristic approaches. Hybrid optimization/heuristic scheduler [40] decomposes the whole model to mapping, routing, and timing phases and adopts phase overlapping and heuristic searching. The above two schedulers are different from CaSMap's multi-level decomposition and non-expertise optimization. HiMap [57] is a scalable RSA mapper with hierarchical software abstraction only for multi-dimensional loop kernels.

FPGA P&R: There are also multi-level optimization techniques in FPGA placement and routing, such as min-cut [24] and hierarchical placement [2]. However, because of the much larger synthesis space and looser constraints compared with RSA mapping, they adopt heuristic partition/placement algorithm whose solution quality is sensitive to the initial random result, different from CaSMap.

Operator/PE Fusion: RADISH [59] proposed *PE fusion* that support domain-specific frequent sub-DFGs to maximize resource sharing. Actually, it is a kind of synthesizing technique technique. Contrarily, our method does not generate PE structures but exploits new abstract layers for optimizing the mapping process. AURORA [49] fuses the DFG operations on the critical path based on the hardware functionality of PEs for throughput optimization. However, it lacks higher-level clustering to optimize the mapping process. Instruction selection for CISC and DSA [4, 48] provides approaches for operator fusion. However, operator fusion is only one specific clustering tool in CaSMap framework, and there are other tools besides operator fusion.

6 CONCLUSION

Reconfigurable spatial architectures (RSAs) are a strong candidate in the design of domain-specific accelerators. Nevertheless, compilation for RSA is difficult because of the mapping problem's NP-Completeness. In this paper, we present CaSMap, an agile mapper framework for various hardware and software of RSAs. We propose strategies for clustering hardware and software IRs and setting up a multi-level mapping model as the foundation of the framework.

Comparing with an open-source waterfall ILP mapper, our experimental results shows that CaSMap is able to reduce 80.5% scale and 98.8% time cost and maintains the same initiation intervals at the same time.

7 ACKNOWLEDGMENTS

We thank the anonymous reviewers for their thoughtful comments. This work is supported in part by National Key R&D Program of China (Grant No. 2018YFB2202101), Beijing Superstring Academy of Memory Technology, National Natural Science Foundation of China (Grant No. 61834002, 62104014), and National Science and Technology Major Project of the Ministry of Science and Technology of China (Grant No. 2018ZX01027101-002).

REFERENCES

- [1] Tobias Achterberg. 2019. What's new in Gurobi 9.0. *Webinar Talk*. url: <https://www.gurobi.com/wp-content/uploads/2019/12/Gurobi-9.0-Overview-Webinar-Slides-1.pdf> (2019).
- [2] S Areibi, G Grewal, D Banerji, and P Du. 2007. Hierarchical FPGA placement. *Canadian Journal of Electrical and Computer Engineering* 32, 1 (2007), 53–64.
- [3] Mahesh Balasubramanian and Aviral Shrivastava. 2020. CRIMSON: Compute-Intensive Loop Acceleration by Randomized Iterative Modulo Scheduling and Optimized Mapping on CGRAs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 11 (2020), 3300–3310.
- [4] Martin Bravenboer and Elco Visser. 2002. Rewriting strategies for instruction selection. In *International Conference on Rewriting Techniques and Applications*. Springer, 237–251.
- [5] Andrew Burgess. 2021. Embench-lot. <https://www.embench.org/>
- [6] Pablo De Oliveira Castro, Chadi Akel, Eric Petit, Mihail Popov, and William Jalby. 2015. Cere: Llvm-based codelet extractor and replayer for piecewise benchmarking and optimization. *ACM Transactions on Architecture and Code Optimization (TACO)* 12, 1 (2015), 1–24.
- [7] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 367–379.
- [8] S Alexander Chin and Jason H Anderson. 2018. An architecture-agnostic integer linear programming approach to CGRA mapping. In *Proceedings of the 55th Annual Design Automation Conference*. 1–6.
- [9] S Alexander Chin, Noriaki Sakamoto, Allan Rui, Jim Zhao, Jin Hee Kim, Yuko Hara-Azumi, and Jason Anderson. 2017. CGRA-ME: A unified framework for CGRA modelling and exploration. In *2017 IEEE 28th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE, 184–189.
- [10] Shail Dave, Mahesh Balasubramanian, and Aviral Shrivastava. 2018. RAMP: resource-aware mapping for CGRAs. In *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [11] Caleb Donovick, Makai Mann, Clark Barrett, and Pat Hanrahan. 2019. Agile SMT-Based Mapping for CGRAs with Restricted Routing Networks. In *2019 International Conference on ReConfigurable Computing and FPGAs (ReConFig)*. IEEE, 1–8.
- [12] John Forrest and Robin Lougee-Heimer. 2005. CBC user guide. In *Emerging theory, methods, and applications*. INFORMS, 257–277.
- [13] Rani Gnanaolivu, Theodore S Norvell, and Ramachandran Venkatesan. 2010. Mapping loops onto coarse-grained reconfigurable architectures using particle swarm optimization. In *2010 International Conference of Soft Computing and Pattern Recognition*. IEEE, 145–151.
- [14] Venkatraman Govindaraju, Chen-Han Ho, Tony Nowatzki, Jatin Chhugani, Nadathur Satish, Karthikeyan Sankaralingam, and Changkyu Kim. 2012. Dyser: Unifying functionality and parallelism specialization for energy-efficient computing. *IEEE Micro* 32, 5 (2012), 38–51.
- [15] Tobias Grosser, Hongbin Zheng, Raghu Aloor, Andreas Simbürger, Armin Größlinger, and Louis-Noël Pouchet. 2011. Polly-Polyhedral optimization in LLVM. In *Proceedings of the First International Workshop on Polyhedral Compilation Techniques (IMPACT)*, Vol. 2011. 1.
- [16] Mahdi Hamzeh, Aviral Shrivastava, and Sarma Vrudhula. 2012. EPIMap: Using epimorphism to map applications on CGRAs. In *Proceedings of the 49th Annual Design Automation Conference*. 1284–1291.
- [17] Kartik Hegde, Po-An Tsai, Sitao Huang, Vikas Chandra, Angshuman Parashar, and Christopher W Fletcher. 2021. Mind mappings: enabling efficient algorithm-accelerator mapping space search. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 943–958.

- [18] Jim Highsmith and Alistair Cockburn. 2001. Agile software development: The business of innovation. *Computer* 34, 9 (2001), 120–127.
- [19] DR Karger, W Lee, and VĖ Mirrokni. 2002. A theoretical and practical approach to instruction scheduling on spatial architectures. (2002).
- [20] Manupa Karunaratne, Aditi Kulkarni Mohite, Tulika Mitra, and Li-Shiuan Peh. 2017. Hycube: A cgra with reconfigurable single-cycle multi-hop interconnect. In *Proceedings of the 54th Annual Design Automation Conference 2017*. 1–6.
- [21] Ryan Kastner. 2005. EXPRESS - Benchmarks. <https://web.ece.ucsb.edu/EXPRESS/benchmark/>
- [22] Takuya Kojima, Nguyen Anh Vu Doan, and Hideharu Amano. 2020. GenMap: A Genetic Algorithmic Approach for Optimizing Spatial Mapping of Coarse-Grained Reconfigurable Architectures. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 28, 11 (2020), 2383–2396.
- [23] Mingyang Kou, Jiangyuan Gu, Shaojun Wei, Hailong Yao, and Shouyi Yin. 2020. TAEM: Fast Transfer-Aware Effective Loop Mapping for Heterogeneous Resources on CGRA. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [24] Chuck Kring and A Richard Newton. 1991. A Cell-Replicating Approach to Minicut-Based Circuit Partitioning.. In *ICCAD*, Vol. 1991. Citeseer, 2–5.
- [25] Chris Lattner. 2008. LLVM and Clang: Next generation compiler technology. In *The BSD conference*, Vol. 5.
- [26] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization*, 2004. CGO 2004. IEEE, 75–86.
- [27] Jinho Lee and Trevor E Carlson. 2021. Ultra-Fast CGRA Scheduling to Enable Run Time, Programmable CGRAs. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1207–1212.
- [28] Yan Li, Tim Callahan, Ervan Darnell, Randolph Harr, Uday Kurkure, and Jon Stockwood. 2000. Hardware-software co-design of embedded reconfigurable architectures. In *Proceedings of the 37th Annual Design Automation Conference*. 507–512.
- [29] Zhaoying Li, Dan Wu, Dhananjaya Wijerathne, and Tulika Mitra. 2022. LISA: Graph Neural Network based Portable Mapping on Spatial Accelerators. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE.
- [30] Dajiang Liu, Shouyi Yin, Leibo Liu, and Shaojun Wei. 2013. Polyhedral model based mapping optimization of loop nests for CGRAs. In *Proceedings of the 50th Annual Design Automation Conference*. 1–8.
- [31] Dajiang Liu, Shouyi Yin, Guojie Luo, Jiaxing Shang, Leibo Liu, Shaojun Wei, Yong Feng, and Shangbo Zhou. 2018. Data-flow graph mapping optimization for CGRA with deep reinforcement learning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 12 (2018), 2271–2283.
- [32] Feng Liu, Heejin Ahn, Stephen R Beard, Taewook Oh, and David I August. 2015. DynaSpAM: Dynamic spatial architecture mapping using out of order instruction schedules. In *2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 541–553.
- [33] Leibo Liu, Zhaoshi Li, Chen Yang, Chenchen Deng, Shouyi Yin, and Shaojun Wei. 2017. Hrea: An energy-efficient embedded dynamically reconfigurable fabric for 13-dwarfs processing. *IEEE Transactions on Circuits and Systems II: Briefs* 65, 3 (2017), 381–385.
- [34] Leibo Liu, Jianfeng Zhu, Zhaoshi Li, Yanan Lu, Yangdong Deng, Jie Han, Shouyi Yin, and Shaojun Wei. 2019. A Survey of Coarse-Grained Reconfigurable Architecture and Design: Taxonomy, Challenges, and Applications. *ACM Comput. Surv.* 52, 6, Article 118 (Oct. 2019), 39 pages. <https://doi.org/10.1145/3357375>
- [35] Wenyan Lu, Guihai Yan, Jiajun Li, Shijun Gong, Yinhe Han, and Xiaowei Li. 2017. Flexflow: A flexible dataflow accelerator architecture for convolutional neural networks. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 553–564.
- [36] Bingfeng Mei, Serge Vernalde, Diederik Verkest, Hugo De Man, and Rudy Lauwereins. 2002. DRESC: A retargetable compiler for coarse-grained reconfigurable architectures. In *2002 IEEE International Conference on Field-Programmable Technology, 2002.(FPT). Proceedings*. IEEE, 166–173.
- [37] Yukio Miyasaka and Masahiro Fujita. 2020. SAT-Based Data-Flow Mapping Onto Array Processor. In *2020 IFIP/IEEE 28th International Conference on Very Large Scale Integration (VLSI-SOC)*. IEEE, 22–27.
- [38] Chris Nicol. 2017. A coarse grain reconfigurable array (CGRA) for statically scheduled data flow computing. *Wave Computing White Paper* (2017).
- [39] Aina Niemetz, Mathias Preiner, and Armin Biere. 2014. Boolector 2.0. *Journal on Satisfiability, Boolean Modeling and Computation* 9, 1 (2014), 53–58.
- [40] Tony Nowatzki, Newsha Ardalani, Karthikeyan Sankaralingam, and Jian Weng. 2018. Hybrid optimization/heuristic instruction scheduling for programmable accelerator codesign. In *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques*. 1–15.
- [41] Tony Nowatzki, Michael Sartin-Tarm, Lorenzo De Carli, Karthikeyan Sankaralingam, Cristian Estan, and Behnam Robatmili. 2013. A general constraint-centric scheduling framework for spatial architectures. *ACM SIGPLAN Notices* 48, 6 (2013), 495–506.
- [42] L. Gurobi Optimization. 2021. "Gurobi optimizer reference manual," 2021.
- [43] Raghu Prabhakar, Yaqi Zhang, David Koeplinger, Matt Feldman, Tian Zhao, Stefan Hadjis, Ardavan Pedram, Christos Kozyrakis, and Kunle Olukotun. 2017. Plasticine: A reconfigurable architecture for parallel patterns. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 389–402.
- [44] B Ramakrishna Rau. 1994. Iterative modulo scheduling: An algorithm for software pipelining loops. In *Proceedings of the 27th annual international symposium on Microarchitecture*. 63–74.
- [45] Winston W Royce. 1970. Managing the Development of Large Software Systems.[Online] <http://www.cs.umd.edu/class/spring2003/cmsc838p/Process/waterfall.pdf> (1970).
- [46] Amirali Sharifian, Reza Hojabr, Navid Rahimi, Sihao Liu, Apala Guha, Tony Nowatzki, and Arrvinth Shriraman. 2019. μ ir-an intermediate representation for transforming and optimizing the microarchitecture of application accelerators. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 940–953.
- [47] Hartej Singh, Ming-Hau Lee, Guangming Lu, Fadi J Kurdahi, Nader Bagherzadeh, and Eliseu M Chaves Filho. 2000. MorphoSys: an integrated reconfigurable system for data-parallel and computation-intensive applications. *IEEE transactions on computers* 49, 5 (2000), 465–481.
- [48] Matthew Sotoudeh, Anand Venkat, Michael Anderson, Evangelos Georganas, Alexander Heinecke, and Jason Knight. 2019. ISA mapper: a compute and hardware agnostic deep learning compiler. In *Proceedings of the 16th ACM International Conference on Computing Frontiers*. 164–173.
- [49] Cheng Tan, Chenhao Xie, Ang Li, Kevin J Barker, and Antonino Tumeo. 2021. AURORA: Automated Refinement of Coarse-Grained Reconfigurable Accelerators. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1388–1393.
- [50] Panagiotis Theocharis and Bjorn De Sutter. 2016. A bimodal scheduler for coarse-grained reconfigurable arrays. *ACM Transactions on Architecture and Code Optimization (TACO)* 13, 2 (2016), 1–26.
- [51] Yatish Turakhia, Sneha D Goenka, Gill Bejerano, and William J Dally. 2019. Darwin-WGA: A co-processor provides increased sensitivity in whole genome alignments with high speedup. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 359–372.
- [52] Matthew JP Walker and Jason H Anderson. 2019. Generic connectivity-based CGRA mapping via integer linear programming. In *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 65–73.
- [53] Chris Walshaw. 2004. Multilevel refinement for combinatorial optimisation problems. *Annals of Operations Research* 131, 1 (2004), 325–372.
- [54] Bo Wang, Manupa Karunaratne, Aditi Kulkarni, Tulika Mitra, and Li-Shiuan Peh. 2019. HyCUBE: A 0.9 V 26.4 MOPS/mW, 290 pJ/op, Power Efficient Accelerator for IoT Applications. In *2019 IEEE Asian Solid-State Circuits Conference (A-SSCC)*. IEEE, 133–136.
- [55] Jian Weng, Sihao Liu, Vidushi Dadu, Zhengrong Wang, Preyas Shah, and Tony Nowatzki. 2020. Dsagen: Synthesizing programmable spatial accelerators. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 268–281.
- [56] Jian Weng, Sihao Liu, Zhengrong Wang, Vidushi Dadu, and Tony Nowatzki. 2020. A hybrid systolic-dataflow architecture for inductive matrix algorithms. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 703–716.
- [57] Dhananiya Wijerathne, Zhaoying Li, Anuj Pathania, Tulika Mitra, and Lothar Thiele. 2021. HiMap: Fast and Scalable High-Quality Mapping on CGRA via Hierarchical Abstraction. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1192–1197.
- [58] Mark Wijnlt, Luc Waeijen, and Henk Corporaal. 2016. Coarse grained reconfigurable architectures in the past 25 years: Overview and classification. In *2016 International Conference on Embedded Computer Systems: Architectures, Modeling and Simulation (SAMOS)*. IEEE, 235–244.
- [59] Max Willsey, Vincent T Lee, Alvin Cheung, Rastislav Bodík, and Luis Ceze. 2018. Iterative search for reconfigurable accelerator blocks with a compiler in the loop. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 3 (2018), 407–418.
- [60] T Wouters, J Yasskin, and C Winter. 2010. unladen-swallow: A faster implementation of python.
- [61] Shouyi Yin, Dajiang Liu, Lifeng Sun, Leibo Liu, and Shaojun Wei. 2017. DFGNet: Mapping dataflow graph onto CGRA by a deep learning approach. In *2017 IEEE international symposium on circuits and systems (ISCAS)*. IEEE, 1–4.
- [62] Shouyi Yin, Xianqing Yao, Dajiang Liu, Leibo Liu, and Shaojun Wei. 2015. Memory-aware loop mapping on coarse-grained reconfigurable architectures. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 24, 5 (2015), 1895–1908.
- [63] Yaqi Zhang, Nathan Zhang, Tian Zhao, Matt Vilim, Muhammad Shahbaz, and Kunle Olukotun. 2021. SARA: scaling a reconfigurable dataflow accelerator. In *Proceedings of the 48th Annual International Symposium on Computer Architecture*. 1041–1054.

[64] Zhongyuan Zhao, Weiguang Sheng, Qin Wang, Wenzhi Yin, Pengfei Ye, Jinchao Li, and Zhigang Mao. 2020. Towards Higher Performance and Robust Compilation

for CGRA Modulo Scheduling. *IEEE Transactions on Parallel and Distributed Systems* 31, 9 (2020), 2201–2219.